Gosu Queries

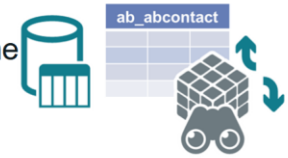
Query basics

Query objects

- A **Gosu query** specifies the entity and criteria associated with the entity to retrieve from the Guidewire application
  - Use Gosu queries to retrieve one or more objects
    - Referenced by foreign key relationships
    - NOT in an array relationship



- **Query object** specifies the query criteria
  - Which entity to query
  - What restrictions to apply
- **Results object** is a set of entity instances fetched from the Guidewire application
  - How many?
  - How to order?



Query objects

Gosu queries are useful when code must work with a set of objects that does not exist as an array, such as **all contact notes created by this user**. Guidewire recommends that all new queries be written as Gosu queries because Gosu queries have the following advantages:

- They more readily support the construction of complex queries
- They use builder patterns, which simplify the creation of search screens
- They use a syntax more intuitive to people accustomed to writing queries in Structured Query Language (SQL)

Results are fetched from the Guidewire application when they are counted, made iterable, or when one object is retrieved. Depending on the query, the results may be fetched from the Guidewire application cache and/or the Guidewire application database. To learn more, refer to documentation in the Gosu Reference Guide.

Steps to execute a basic query

1. Create the query object using the `make` method
2. Create the results object using the `select` method
3. Process the results of the query as needed

Step 1: Create the query object

```
1  uses gw.api.database.Query
2
3  var output = ""
4  var queryObj = Query.make(entity.ABContact)
```

- `Query` is a class in the **gw.api.database** package
- `make(EntityName)` is a method that requires a named entity type

Notes

Step 1: Create the query object

Code example
Line 4
- Creates the query object for the ABContact entity type.
- The query retrieves all inst
- ances of ABContact.
- **Best practice** would dictate that the query above should contain some restrictions, as the ABContact entity may contain many records. Restrictions are covered later in this lesson.

Step 2: Create the results object

```
1  uses gw.api.database.Query
2
3  var output = ""
4  var queryObj = Query.make(entity.ABContact)
5  var resultsObj = queryObj.select()
```

- Create the result object with select method
  - However, results are not fetched until they are referenced

Notes

Step 2: Create the results object

Code example
Line 5
- Creates the results object.

In the slide example, the results object is a convention the helps distinguish the query object and the results object. The results object is not required. It is not always necessary to create an explicit results object.
For example, it is possible to write the above as:

var queryObj = Query.make(ABContact).select()

Step 3: Process results



- Use a `for` loop to iterate through the result set
- DisplayName property improves the readability of the console output

Step 3: Process results

Code example
Lines 7-9
- Fetches the result objects

Line 10
- Prints to console

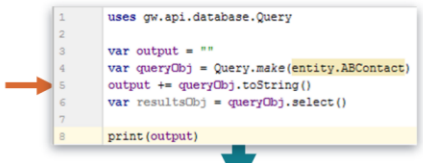
Recall the for-loop syntax in Gosu has an optional index variable. Use the index variable to display a number next to each result:

```
for(anABContact in resultsObj
index i) {
print((i+1) + ": " +
anABContact.DisplayName)
}
```

…produces the following output...
1: Stan Newton
2: Bo Simpson
3: Bo Simpson
4: William Weeks

Viewing approximation of SQL query

- `toString` approach



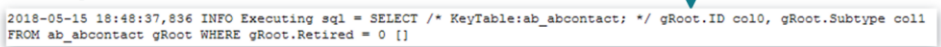
  - Output from console



- `withLogSQL` approach



  - Output from log



Viewing approximation of SQL query

The query builder APIs provide two ways to preview and record SQL SELECT statements that your Gosu code and the query builder APIs submit to the application database:

toString – provides an approximation of the SQL Select statement before it is submitted. In the example above:

- The first line shows square brackets ([]) with list of variables to bind to the query. In this case, there are none.
- The remaining lines show the SQL statement. Note that the table for the entity type, ab_abcontact, has the table alias gRoot.

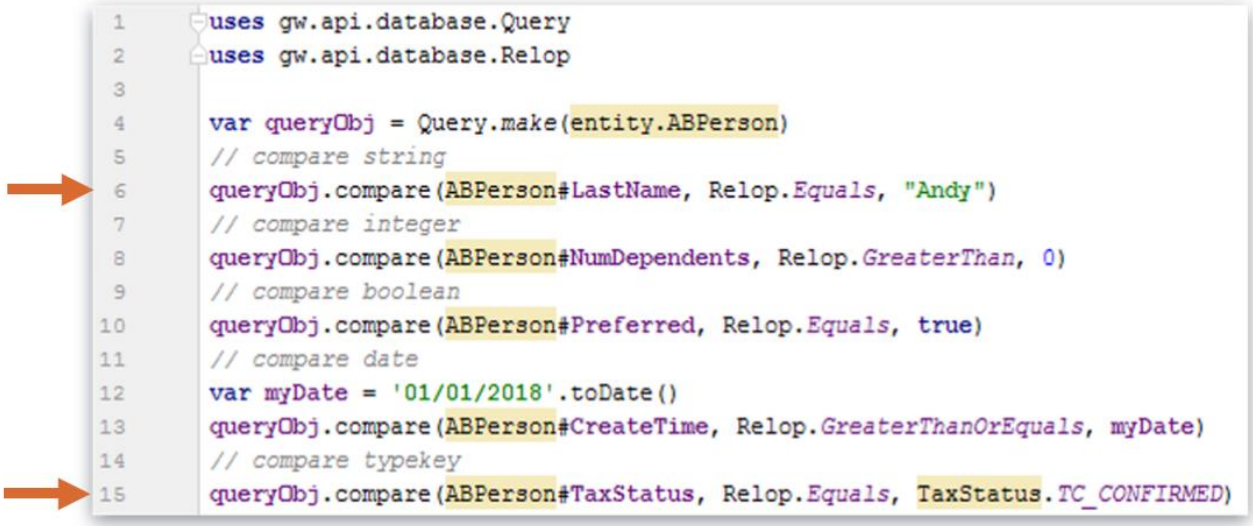
withLogSQL – displays and records the exact SQL Select statement at the time it is submitted. Writing to the system logs and to standard output does not occur when Gosu sample code calls the withLogSQL method. That logging occurs some time later, when the application submits the query to the relational database. These results are an approximation because the actual SQL may have vendor-dependent variations or vendor-dependent optimizations or both. A given query may have slight syntactic variations depending on whether it is used for an Oracle database or a SQL Server database.

Query restrictions

`compare` method



- Comparison predicates are columns in SQL Where clause
  - `queryObj.compare(type#Feature, operator, value)`
- When applied to string fields, the values in the field must match exactly the comparison value
- `compare` method is overloaded, meaning it has one or more method signatures



```
1   uses gw.api.database.Query
2   uses gw.api.database.Relop
3
4   var queryObj = Query.make(entity.ABPerson)
5   // compare string
6   queryObj.compare(ABPerson#LastName, Relop.Equals, "Andy")
7   // compare integer
8   queryObj.compare(ABPerson#NumDependents, Relop.GreaterThan, 0)
9   // compare boolean
10  queryObj.compare(ABPerson#Preferred, Relop.Equals, true)
11  // compare date
12  var myDate = '01/01/2018'.toDate()
13  queryObj.compare(ABPerson#CreateTime, Relop.GreaterThanOrEquals, myDate)
14  // compare typekey
15  queryObj.compare(ABPerson#TaxStatus, Relop.Equals, TaxStatus.TC_CONFIRMED)
```

Notes

compare method

Code example
Line 6
- String literals require quotes

Line 15
- Use typekey reference syntax for typecode comparisons

Valid operators include:
- Relop.Equals
- Relop.NotEquals
- Relop.LessThan
- Relop.LessThanOrEquals
- Relop.GreaterThan
- Relop.GreaterThanOrEquals

Gosu provides typekey literals that let you specify typelist codes in your programming code. Gosu creates typekey literals at compile time by prefixing typelist codes with TC_ and converting code values to upper case. Avoid strings for typecode values as this also requires casting to a typekey.

`null` values for query restrictions

```
1   uses gw.api.database.Query
2   uses gw.api.database.Relop
3
4   var queryNull = Query.make(entity.ABPerson)
5   var queryNotNull = Query.make(entity.ABPerson)
6
7   // ABPerson where Score IS NULL
8   queryNull.compare(ABPerson#Score, Relop.Equals, null)
9
10  // ABPerson where Score IS NOT NULL
11  queryNotNull.compare(ABPerson#Score, Relop.NotEquals, null)
```

- Entities can have elements that allow for null values
- If there is no value, the database leaves the field as null unless the element specifies a default value
- Restriction queries can compare for null or not null values

gw.api.database.Query	ANSI SQL
compare(entity.Property, Equals, null)	where table.columnName IS NULL
compare(entity.Property, NotEquals, null)	where table.columnName IS NOT NULL

Notes

null values for query restrictions

In Guidewire applications, there are some data model entity elements that specify a nullok attribute. Column, foreignkey, and typekey are elements that have the nullok attribute. When specified as an attribute of an element, you must define the nullok attribute. A null value signifies the absence of or void of a value.

Guidewire applications transform an entity element with a nullok=true attribute to a column definition in a physical database table. Relational databases create database tables using a form of Structure Query Language (SQL) known as Data Definition Language (DDL). With a nullok=false attribute, a Guidewire application executes a SQL / DDL statement to create a column constraint that specifies that the named column cannot contain a null value.

Multiple restrictions ANDed together

```
1  uses gw.api.database.Query
2  uses gw.api.database.Relop
3
4  var output = ""
5  var queryObj = Query.make(ABPerson)
6  // Last name
7  queryObj.compare(ABPerson#LastName, Relop.Equals, "Andy")
8  // First name
9  queryObj.compare(ABPerson#FirstName, Relop.Equals, "William")
```

Restrictions are inherently ANDed together

Multiple restrictions ANDed together

Code example
Line 7
- Restricts ABPerson query to last name equal to Andy

Line 9
- Restricts ABPerson query to first name equal to William

SQL equivalent:

SELECT *
FROM ABPerson
WHERE ABPerson.LastName = "Andy"
AND ABPerson.FirstName = "William"

A query can have as many restrictions as needed.

Multiple restrictions ORed together

```
1  uses gw.api.database.Query
2  uses gw.api.database.Relop
3
4  var output = ""
5  var queryObj = Query.make(ABPerson)
6  // Last name or First name
7  queryObj.or(\ criteria -> {
8    criteria.compare(ABPerson#LastName, Relop.Equals, "Andy")
9    criteria.compare(ABPerson#FirstName, Relop.Equals, "William")
10 })
```

`or` method requires block that specifies criteria to OR together

Multiple restrictions ORed together

Code example
Lines 7-10

- **or** method block consists of a criteria element that represents the query object.

You can create a query that consists of multiple conditions that are ORed together. To do this, you must use the **or** method. It takes a block as an input parameter. The block consists of a placeholder that represents the original query object. You can then list multiple criteria, all of which are ORed together.

The block syntax for the or method is:

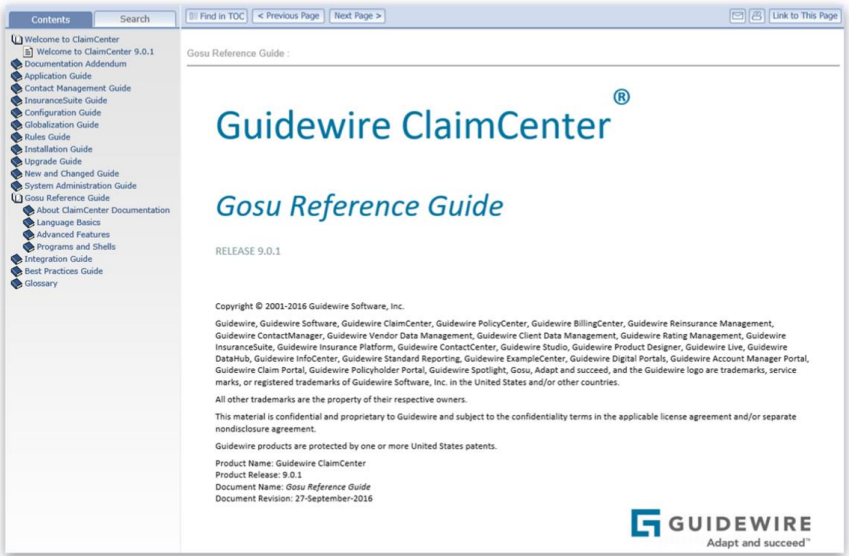
```
queryObj.or( \ criteria -> {
criteria.compare
criteria.compare
// add as many criteria as
needed
})
```

You can also combine conditions with AND and OR logic in complex ways. The following example finds all policy people whose first name is "John" or whose name is "Erica Hinds" (first name is Erica and last name is Hinds):

```
policyPersonQuery.or( \
orCriteria -> {
orCriteria.compare(ABPers
on#FirstName,
Relop.Equals, "John")
orCriteria.and( \
andCriteria -> {
andCriteria.compare(ABPe
rson#FirstName,
Relop.Equals, "Erica")
andCriteria.compare(ABPe
rson#LastName,
Relop.Equals, "Hinds")
})
})
```

Additional restriction options

- See the **Predicate Methods Reference** in the Gosu Reference Guide:
  - `between`
  - `compareIgnore`
  - `contains`
  - `subselect`
  - `startsWith`

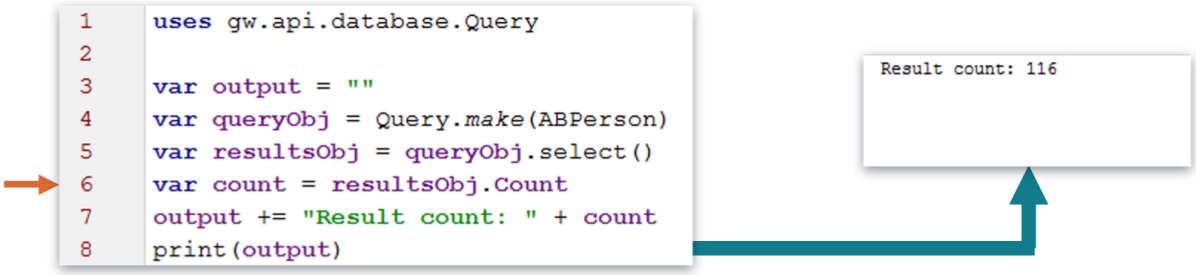


Additional restriction options

The Gosu Reference Guide contains complete list of all options for restricting queries. In addition, documentation covers how to write aggregate queries, row queries, column value comparison queries, and join queries.

Working with query results

Counting results



- Other methods:
  - **`resultsObj.HasElements`** (uses Count under the covers)
  - **`resulstsObj.iterator.hasNext`** (performs very poorly)
- Depending on the requirement, there may be better performing alternatives…

Counting results

Code example
Line 6
- resultsObj.Count identifies the number objects in the result set.

Developers should keep in mind that Guidewire does not query the database for the complete result set, however, and does not store a snapshot of the results of a given query. Therefore, the value of Count can vary if the number of rows in the database that meet the query's criteria changes while the query results are being processed. The field is introduced here as it is useful for instructional and debugging purposes, but it can be problematic if used in implementation code as an absolutely accurate reflection of the number of rows in the database that meet the query's criteria. For a further discussion of this topic, refer to the Gosu Reference Guide. The next slide discusses some alternatives to Count that, depending on the requirement will work equally well and provide superior performance.

Taking a peek without retrieving any data



- `.Count` is often used to answer questions such as:
  - Will this query return anything?
  - Will this query return too much?
- These methods perform better and are more reliable than `.Count` :
  - **`isEmpty`** does fast check to see if query has any results
  - **`getCountLimitedBy(x)`** returns number if results if less than x, otherwise x
- **Peeking** is a waste of time and cycles if you intend to process the entire result set

Taking a peek without retrievin any data

Will query return anything?
- To control whether to show/hide UI element, or proceed with computation

Will query return too much?
- To decide whether to go ahead with something potentially expensive, e.g. search screens, tabs

isEmpty and getCountLimitedBy(x) perform better and more reliable than .Count. You won't see any difference on H2 but on SQLServer or Oracle they will issue a count query with an extra clause that tells the database only the first n results are relevant (where n is 1 for isEmpty, and the value passed in for getCountLimitedBy(x)). This allows the database query optimizer to only find that number of matching rows before bailing out, which can make a significant difference if a large number of rows are involved.

Querying for one result

```
1  uses gw.api.database.Query
2  uses gw.api.database.Relop
3
4  var output = ""
5  var queryObj = Query.make(ABCompany)
6  queryObj.compare(ABCompany#Name, Relop.Equals, "Albertson's")
7  var resultsObj = queryObj.select().AtMostOneRow
8  output += resultsObj.EmailAddress1
9  print(output)
```

```
info@Albertsons.com
```

`AtMostOneRow` is a property that…

- If a single item exists, returns that single row
- If no item exists, returns null
- If multiple items exist, throws an exception

Querying for one result

AtMostOneRow is annotated as a property, but it is just the published name for the getAtMostOneRow method.

Sorting results

- Ordering methods all take an object that implements the **`IQuerySelectColumns`** interface in the package as their one argument
  - The **`Paths.make`** method yields an object access path from the primary entity to a simple, foreign key, database-backed property
  - Access path cannot include virtual properties, methods, or calculations

```
1  uses gw.api.database.Query
2  uses gw.api.database.QuerySelectColumns
3  uses gw.api.path.Paths
4
5  var queryObj = Query.make(ABPerson)
6  var resultsObj = queryObj.select()
7  // sort results by state
8  resultsObj.orderBy(QuerySelectColumns.path(Paths.make(ABPerson#PrimaryAddress, Address#State)))
9  // then, sort results by last name
10 resultsObj.thenBy(QuerySelectColumns.path(Paths.make(ABPerson#LastName)))
11 // then, sorts results by first name
12 resultsObj.thenBy(QuerySelectColumns.path(Paths.make(ABPerson#FirstName)))
13
14 for(anPerson in resultsObj) {
15   print(anPerson.PrimaryAddress.State + ": " + anPerson.DisplayName)
16 }
```

```
AL: Marc Lauria
AL: Adelle Lynn
AL: Lorine Mccallion
AZ: Alexa Purves
CA: James Andersen
CA: Samantha Andrews
CA: Eric Andy
CA: William Andy
CA: Stefani Arrowsmith
CA: Domonique Dames
CA: John Dan
CA: William Dan
CA: Harper David
CA: William David
```

Sorting results

Code example
Line 8
- Path object traverses foreign key and ends with State property
- In this case, the foreign key name does not match the entity name

Lines 10,12
- Multiple levels of ordering query results

The Query builder APIs support the following ordering methods on result objects.

METHOD	DESCRIPTION
orderBy	Clears all previous ordering, then orders results by the specified column in ascending order.
orderByDescending	Clears all previous order methods, and orders by the specified column in descending order.
thenBy	Orders by the specified column in ascending order. Does not clear previous ordering.
thenByDescending	Orders by the specified column in descending order. Does not clear previous ordering.

Filter entities on database

- Return only what you need
- Entities can use up memory and processing time
- Request the database only for entities that are essential to the current action
  - Perform filtering at the database rather than in memory
  - Avoid where() method
  - Avoid first(), last(), firstWhere(), and lastWhere() methods
- Use dot notation carefully

Filter entities on database

Next, we are going to focus on a few anti-patterns related to filtering entities.
Best practice guidelines:

- Return only what you need, because entities can use up a lot of memory and processing time.
- Ask the database only for the entities that are essential to the current action.
- Let the database do what it is good at, such as filtering.
- Avoid the methods *where(), first(), last(), firstWhere(), and lastWhere()*.

It is important to keep in mind that the application can run implicit queries behind the scenes and add related entities to a bundle when developers use dot notation to access foreign key and array fields. Use caution and code carefully when using these fields, because bundle sizes can grow extremely large and negatively affect performance. On the next few slides we will examine these anti-patterns one by one.

Filter at the database rather than in memory

- Anti-Pattern

```
4   var targetPolicy : Policy
5   // Performance suffers because
6   // the query loads all claims
7   var claimQuery = Query.make(Claim)
8   for (claim in claimQuery.select()) {
9     if (claim.Policy == targetPolicy) {
10      // Do something on claims of targeted policies.
11      //...
12    }
13  }
```

- Best Practice

```
4   var targetPolicy : Policy
5
6   var claimQuery = Query.make(Claim)
7   // Query comparison method filters claims on database
8   claimQuery.compare(Claim#Policy, Equals, targetPolicy)
9   for (claim in claimQuery.select()) {
10    // Do something on claims of targeted policies
11    // ...
12  }
```

Filter at the database rather than in memory

The first anti-pattern in this section is about filtering in memory instead of in the database. The first code example on the slide brings unneeded entities to the application server. It loads every claim into memory and tries to filter the claims based on a target policy. This results in adding every claim to the bundle, which can seriously affect performance because of the potentially large number of claims in the database. Keep in mind that the dot Policy property access in line 9 will also add the policy entities for every claim into the bundle, effectively doubling the number of entities in the bundle. This is clearly an anti-pattern. You should let the database do the filtering and bring back only the entities that you need.

Avoid where()



- Anti-Pattern

```
5  var resultContainer = Query.make(Address)
6      .select().toCollection().where(
7              \ elt -> elt.PostalCode == "90292-6043" )
```

- Best Practice

```
10 var queryObject = Query.make(Address)
11 // Apply qualifying where clause
12 queryObject.compare(Address#PostalCode, Equals, "90292-6043")
13 var selectObject = queryObject.select()
14 var resultIterator = selectObject.iterator()
15 for (address in resultIterator) {
16    // do something with each address
17 }
```

Avoid where()

This anti-pattern involves a similar problem. The first expression does the filtering on the application server by fetching all addresses into a collection, and then making a second collection with the qualifying addresses. This results in the same memory, bundle, and processing time issues as in the previous example.

The second code sample shows the fix for this anti-pattern, which is letting the database filter the results. Instead of loading all the addresses, we use the compare() function in line 12. This limits the results returned from the database. As a best practice, try to avoid *where()* when you can do the filtering as part of the query.

Avoid first(), last(), firstWhere(), and lastWhere()

- Anti-Pattern

```
// Query for addresses in Chicago
var queryAddresses = Query.make(Address)
var selectAddressesList = queryAddresses.select().toList()

// FOR firstWhere(): Sort by Postal Code in ascending order
selectAddressesList.sortBy(\ addr -> addr.PostalCode)

var firstAddress = selectAddressesList.firstWhere(\ addr -> addr.City == "Chicago")

// FOR lastWhere(): Sort by Postal Code in descending order
selectAddressesList.sortByDescending(\ addr -> addr.PostalCode)
var lastAddress = selectAddressesList.lastWhere(\ addr -> addr.City == "Chicago")
```

- Anti-Pattern

```
// Query for addresses in Chicago
var queryAddresses = Query.make(Address)
queryAddresses.compare( Address#~~CityDenorm~~, Equals, "Chicago")
var selectAddressesList = queryAddresses.select().toList()

// FOR first(): Sort by PostalCode in ascending order...
selectAddressesList.sortBy( \ addr -> addr.PostalCode)
var firstAddress = selectAddressesList.first()

// FOR last(): Sort by PostalCode in descending order...
selectAddressesList.sortByDescending(\ addr -> addr.PostalCode)
var lastAddress = selectAddressesList.last()
```

Education

DO NOT DISTRIBUTE
© 2021 Guidewire Software, Inc. For information about Guidewire's trademarks, visit http://guidewire.com/legal-notices.

Avoid first(), last(), firstWhere(), and lastWhere()

Two examples are provided for this next filtering anti-pattern. The expression on the top does the filtering on the application server, needlessly fetching all addresses into a collection and creating a second collection with the qualifying addresses, just to get *one* of those qualifying addresses.

The expression on the bottom does some filtering on the database, but still brings back an entire array of results to get just *one* of the qualifying addresses.

Avoid first(), last(), firstWhere(), and lastWhere() (continued)

- Best Practice

```
uses gw.api.database.Query
uses gw.api.database.QuerySelectColumns
uses gw.api.path.Paths
```

```
// Query for addresses in Chicago
var queryAddresses = Query.make(Address)
queryAddresses.compare( Address#~~CityDenorm~~, Equals, "Chicago")
var selectAddresses = queryAddresses.select()

// FOR first(): Sort the result set
var firstAddress = selectAddresses.orderBy(QuerySelectColumns.path(
    Paths.make(Address#PostalCode))).FirstResult

// FOR last(): Sort the result set in DESCENDING ORDER.
var lastAddress = selectAddresses.orderByDescending(QuerySelectColumns.path(
    Paths.make(Address#PostalCode))).FirstResult
```

Avoid first(), last(), firstWhere(), and lastWhere() (continued)

The fix for this anti-pattern employs three best practices:

First, use the *compare()* predicate to filter the addresses in the database.
Second, when sorting, database-side ordering methods including *orderBy* and *orderByDescending()* to sort the data in the database instead of *sort()*, which has more overhead and does not perform as well. For a full list of ordering methods, refer to the *Gosu Reference Guide* in the InsuranceSuite product documentation.
Finally, access the *FirstResult* property of the *select* object, which returns just the first entry in the result set.

Do not use the *first()* and *last()* methods to check for the existence of entities in the database. The best practice for checking for existence is covered in a later slide.

Count entities on database

- Let the database count entities
  - Avoid countWhere()
  - Avoid toCollection()
  - Avoid iteration over entities from database to count

Count entities on database

Next, we are going to focus on a few anti-patterns related to counting entities. Just as was stated for filtering entities, let the database do what it is good at, which includes counting. Avoid the methods *countWhere(), toCollection(), and select().Count*. Also avoid iterating over entities from the database to count them and counting before accessing entities. On the next few slides we will examine these anti-patterns one by one.

Avoid countWhere()

- Anti-Pattern

```
4   var claim : Claim
5
6   claim.Activities.countWhere(\ a ->
7     a.ActivityPattern.Code == "notification"
8       && a.Status == ActivityStatus.TC_OPEN)
```

- Best Practice

```
11  var claim : Claim
12  var actPattern : ActivityPattern
13
14  var count = Query.make(Activity)
15      .compare(Activity#ActivityPattern, Equals, actPattern)
16      .compare(Activity#Status, Equals, ActivityStatus.TC_OPEN)
17      .join(Activity#Claim)
18      .compare(Claim#ClaimNumber, Equals, claim.ClaimNumber)
19      .select().Count
```

Avoid countWhere()

Let's look at another example. The first code sample implicitly brings back the entire activity array and counts the results that meet the criteria. You do not need any of the actual activities; you just need to know how many satisfy the where criteria. Instead of accessing the Activities array, you can query the target activities and count them in the database using the *Count* property of the results object. When the *Count* property is accessed, the Query API issues a *select count(*)* query, so only the actual count is returned.

Avoid toCollection()

- Anti-Pattern

```
6   var count = Query.make(Claim).select().toList().Count
```

- Best Practice

```
4   var count = Query.make(Claim).select().Count
```

Avoid toCollection()

This example is very similar to the previous one. The *toList()* method explicitly brings all the claims back into a collection and counts them on the application server, even though you do not need the actual entities. You just need to know how many there are.

Avoid iteration over entities from database to count

- Anti-Pattern

```
4   var results = Query.make(Claim).select()
5   var nCount: int = 0
6   // loads and iterates through every claim
7   for (c in results.iterator()) {
8      nCount = nCount + 1
9   }
10
11  return nCount
```

- Best Practice

```
4   var count = Query.make(Claim).select().Count
```

Avoid iteration over entities from database to count

Here is another count example. The first code sample returns the query results to the application server and iterates through every claim, using lots of memory and processing time. In the second code sample, the *Count* property of the select results performs the count in the database and is the most performant way to get that value. Remember the golden rule - bring back only what you actually need.

Which approach is used to record a more specific version of the SQL for a query?

A. queryObj.toString

B. queryObj.withLogSQL

The correct answer is B. queryObj.withLogSQL(true)

What is the difference between a query object and results object?

A) A query object specifies the entity and the query criteria.

B) A results object is a set of one or more entity instances fetched from the Guidewire application.

C) This distinction is helpful for understanding the concept of a query and of set of results. In practice, query results are fetched from the Guidewire application when they are counted, iterated over, or when one object is retrieved.

Which query restriction uses a block of compare methods?

The **OR** restriction uses the or method with a block of compare methods such as:

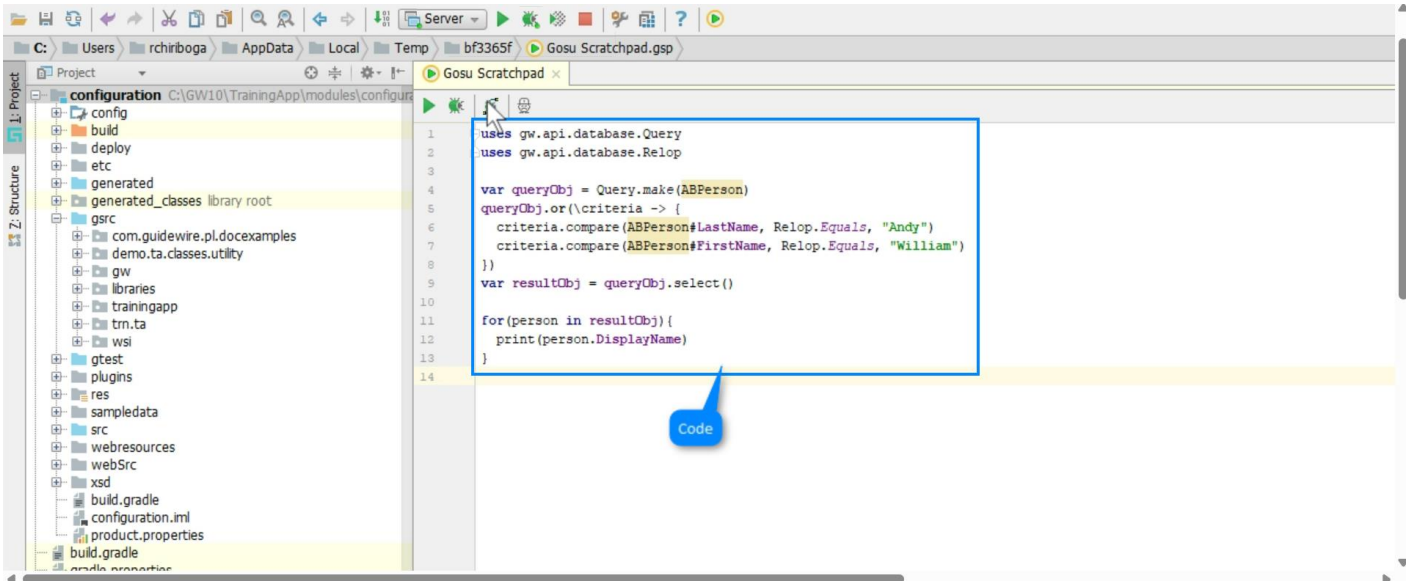
```
var queryObj = Query.make(ABCompany).withLogSQL(true)
queryObj.or(\ criteria -> {
        criteria.compare(ABCompany#Score, Relop.LessThan, 50 )
        criteria.compare(ABCompany#InspectionRequired, Relop.Equals, true)
})
```

What happens if you use the `AtMostOneRow` property in a query, and the query returns more than one object?

An exception is thrown when a query that uses the **`AtMostOneRow`** property, returns more than one object.

Demonstration


Project
configuration C:\GW10\TrainingApp\modules\configura
config
build
deploy
etc
generated
generated_classes library root
gsrc
com.guidewire.pl.docexamples
demo.ta.classes.utility
gw
libraries
trainingapp
trn.ta
wsi
gtest
plugins
res
sampledata
Gosu Scratchpad
Code
uses gw.api.database.Query
uses gw.api.database.Relop
var queryObj = Query.make(ABPerson)
queryObj.compare(ABPerson#LastName, Relop.Equals, "Smith")
var resultObj = queryObj.select()
for(person in resultObj){
print(person.DisplayName)
}


Server
C: Users rchiriboga AppData Local Temp bf3365f Gosu Scratchpad.gsp
Project
Gosu Scratchpad
configuration C:\GW10\TrainingApp\modules\configura
config
build
deploy
etc
generated
generated_classes library root
gsrc
com.guidewire.pl.docexamples
demo.ta.classes.utility
gw
libraries
trainingapp
trn.ta
wsi
gtest
plugins
res
sampledata
src
webresources
webSrc
xsd
build.gradle
configuration.iml
product.properties
build.gradle
uses gw.api.database.Query
uses gw.api.database.Relop
var queryObj = Query.make(ABPerson)
queryObj.or(\criteria -> {
criteria.compare(ABPerson#LastName, Relop.Equals, "Andy")
criteria.compare(ABPerson#FirstName, Relop.Equals, "William")
})
var resultObj = queryObj.select()
for(person in resultObj){
print(person.DisplayName)
}
Code